# Day 1: Introduction to Containers & Google Cloud

---

### 1. Overview of Modernization Trends

**Definition**

Modernization = Transforming legacy applications (monolithic, on-premise) into cloud-native, scalable, and resilient systems.

---

**Key Trends**

**1. Monolith → Microservices**

- Monolith: Single large application
- Microservices: Independent, loosely coupled services

Example:

- E-commerce app:
    - Monolith → One codebase
    - Microservices → User Service, Payment Service, Inventory Service

---

**2. On-Prem → Cloud**

- Physical servers → Cloud platforms (GCP, AWS, Azure)

Benefit:

- No hardware management
- Pay-as-you-go

---

**3. VM-based → Container-based**

- Heavy VMs → Lightweight containers

---

**4. DevOps & CI/CD**

- Faster deployments using:
    - Jenkins
    - GitHub Actions

- Cloud Build

---

**Real-World Case Study**

Netflix Migration

- Moved from data centres to cloud
- Broke monolith into microservices
- Uses containers and Kubernetes

Result:

- Handles millions of users globally
- Auto-scales during peak load

---

**2. Containers vs Virtual Machines**

**Definitions**

Virtual Machine (VM)

- Full OS + app + dependencies
- Runs on hypervisor

Container

- Lightweight package with app + dependencies
- Shares host OS kernel

---

**Comparison Table**

| **Feature** | **VM** | **Container** |
|---|---|---|
| OS | Full OS | Shared OS |
| Size | GBs | MBs |
| Boot Time | Minutes | Seconds |
| Performance | Slower | Faster |
| Isolation | Strong | Process-level |

**Example**

Running a Python App:

- VM:
  - Install OS → Python → dependencies → app
- Container:
  - Pre-built image → run instantly

---

**Scenario**

Startup Scaling Issue

- Traffic spike during sale

Using VMs:

- Takes minutes to spin up
- Expensive

Using Containers:

- Spin up in seconds
- Auto-scale using Kubernetes

---

**Case Study**

Spotify

- Migrated from VMs to containers
- Uses Kubernetes

Result:

- Faster deployments
- Reduced infrastructure cost

---

**3. Docker Fundamentals**

**Definition**

Docker = Platform to build, package, and run applications in containers

---

**Core Components**

**1. Docker Image**

- Blueprint of container

Example:

- Python + Flask + application code

---

**2. Docker Container**

- Running instance of image

---

**3. Dockerfile**

- Script to build image

Example:

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install flask
CMD ["python", "app.py"]
```

---

**4. Docker Hub / Artifact Registry**

- Store images

---

**Workflow**

1. Write Dockerfile
2. Build image
3. Push to registry
4. Run container

---

**Example**

```
docker build -t myapp .
```

```
docker run -p 5000:5000 myapp
```

---

**Scenario**

Works on My Machine Problem

Developer A:

- App works locally

Developer B:

- Fails due to missing dependencies

Solution:

- Use Docker to ensure the same environment everywhere

---

**Case Study**

PayPal

- Uses Docker containers

Benefit:

- Consistent deployments across environments

---

**4. Introduction to Google Cloud (GCP Architecture)**

**Definition**

Google Cloud Platform (GCP) = Suite of cloud services for:

- Compute
- Storage
- Networking
- AI/ML

---

**GCP Architecture Layers**

**1. Global Infrastructure**

- Regions
- Zones

Example:

- Region: asia-south1
- Zones: asia-south1-a, asia-south1-b, asia-south1-c

---

**2. Core Services**

| Category | Services |
|---|---|
| Compute | Compute Engine, GKE, Cloud Run |
| Storage | Cloud Storage |
| Database | BigQuery, Cloud SQL |
| Networking | VPC |

---

**Example**

Deploying an application:

- Compute → GKE
- Storage → Cloud Storage
- Database → Cloud SQL

---

**Scenario**

Multi-region deployment

- Application deployed in:
  - India
  - US

Benefit:

- Low latency
- High availability

---

**Case Study**

Snapchat

- Uses GCP for infrastructure

Result:

- Scales to millions of users

---

**5. IAM & GCP Resource Hierarchy**

**IAM Definition**

IAM (Identity and Access Management) = Controls who can access what

---

**Key Concepts**

1. Identity
    - User / Service Account
2. Role
    - Collection of permissions
3. Policy
    - Binding of identity and role

---

**Resource Hierarchy**

Organization

↓

Folder

↓

Project

↓

Resources

---

**Example**

- Organization → Company
- Folder → Dev / Prod
- Project → GKE Project

- Resource → VM / Bucket

---

**Scenario**

Secure Access

- Dev team → access only Dev project
- Admin → access all

Solution: Use IAM roles

---

**Case Study**

Enterprise Setup

- Folder: Finance
- Project: Billing Application

Result:

- Only finance team has access

---

**6. Hands-On (Trainer Demo Flow)**

---

**1. Install Docker**

Windows (PowerShell):

```
docker --version
```

---

**2. Run First Container**

```
docker run hello-world
```

---

**3. Build Sample Application**

```
docker build -t flask-app .
docker run -p 5000:5000 flask-app
```

---

**4. GCP Project Setup**

```
gcloud init
gcloud projects create my-first-project
gcloud config set project my-first-project
```

---

**5. Enable APIs**

```
gcloud services enable container.googleapis.com
```

---

**6. Authenticate**

```
gcloud auth login
```

---

**Mini Hands-On Scenario**

Task:

- Build Docker image
- Push to Artifact Registry
- Run container

---

**End-to-End Case Study (Day 1 Summary)**

**Scenario: E-Commerce Modernization**

Problem:

- Monolithic application
- Slow deployments
- Scaling issues

---

**Solution**

1. Break into microservices
2. Containerize using Docker
3. Deploy on GKE
4. Secure using IAM

---

**Result**

- Faster deployments
- Auto scaling
- Reduced cost